VOLE-PSI: Fast OPRF and Circuit-PSI from Vector-OLE

Peter Rindal, Phillipp Schoppman

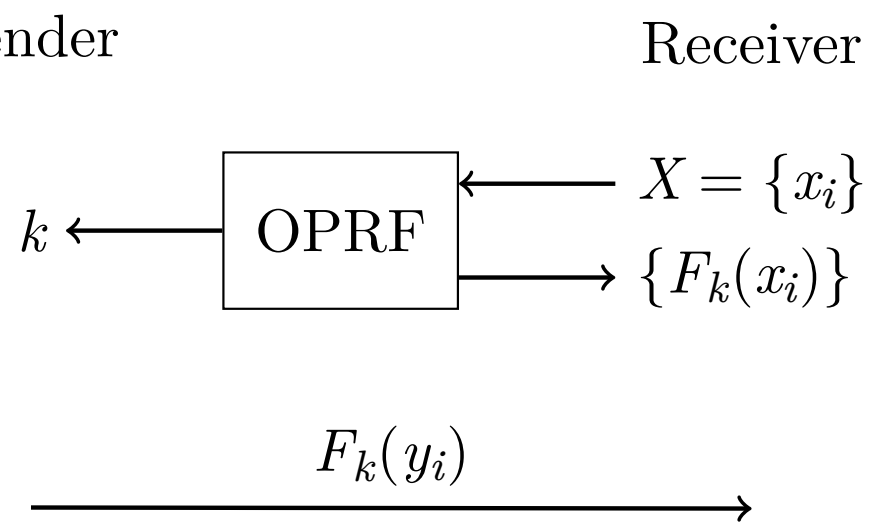
EUROCRYPT 2021

Boyudong Zhu

December 16, 2025

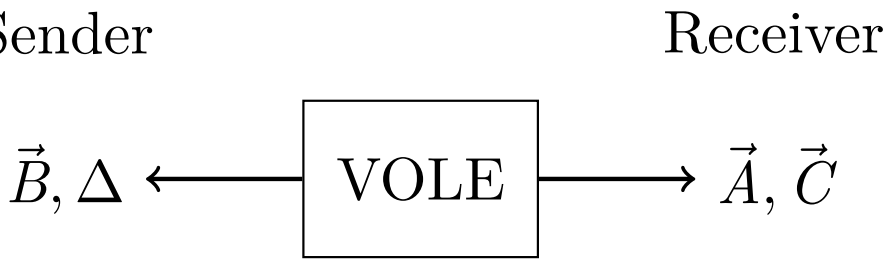
Outline

Oblivious PRF based PSI



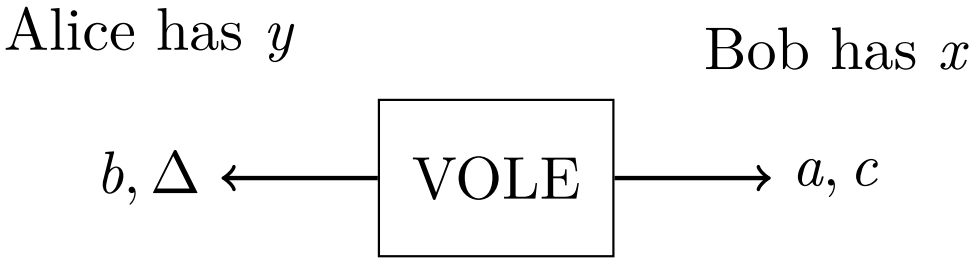
- Sender doesn't know the value and x_i
- Receiver doesn't know the key k
- if $F_k(y_i) \in \{F_k(x_i)\}$, $y_i \in X \cap Y$

Vector OLE



- $\vec{A}, \vec{B}, \vec{C} \in F^m, \Delta \in F$
- $\vec{C} = \Delta\vec{A} + \vec{B}\ ;\ c = \Delta a + b$
- Linear mask
- offline

VOLE and OPRF



When $a = 0$: we can let OPRF-key$=b = c$.
And then we have: if $x = y$ $F_c(x) = F_b(y)$.

- Bob can't choose a.
- $\vec{A}, \vec{B}, \vec{C}$ will as large as domain.
- Bob can't know the key!

How to construct an OPRF from VOLE

Problem 1: Bob can't choose a.

- Bob can choose p. Alice's new key is $k = \Delta a' + b = c + \Delta p$. That is $K = \Delta A' + B$.

Problem 2: $\vec{A}, \vec{B}, \vec{C}$ will as large as domain.

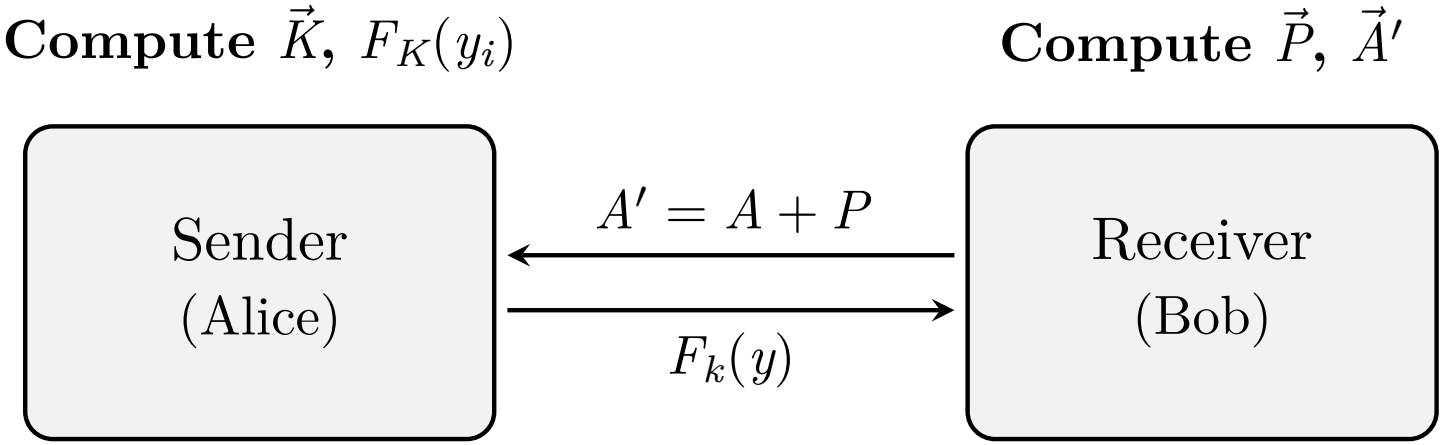
- We use $row(x_i, r)$ to encode X as the matrix $\mathbf{M_x} \in \mathbb{F}^{|X| \times m}$. Then choose $\mathbf{P} \in \mathbb{F}^m$ such that $\mathbf{M_x P} = 0$ for all of Bob's inputs $x_i \in X$, and use a VOLE correlation of size m.And sends $A' = A + P$ to Bob.
- Alice also computes $K = \Delta A' + B$ like Problem 1.
- Alice has $\mathbf{M_y}$: $\mathbf{M_y} K = (\mathbf{M_y} C + \Delta \mathbf{M_y} P) = \mathbf{M_y} C$.

How to construct an OPRF from VOLE

Problem 3: Bob can't know the key.

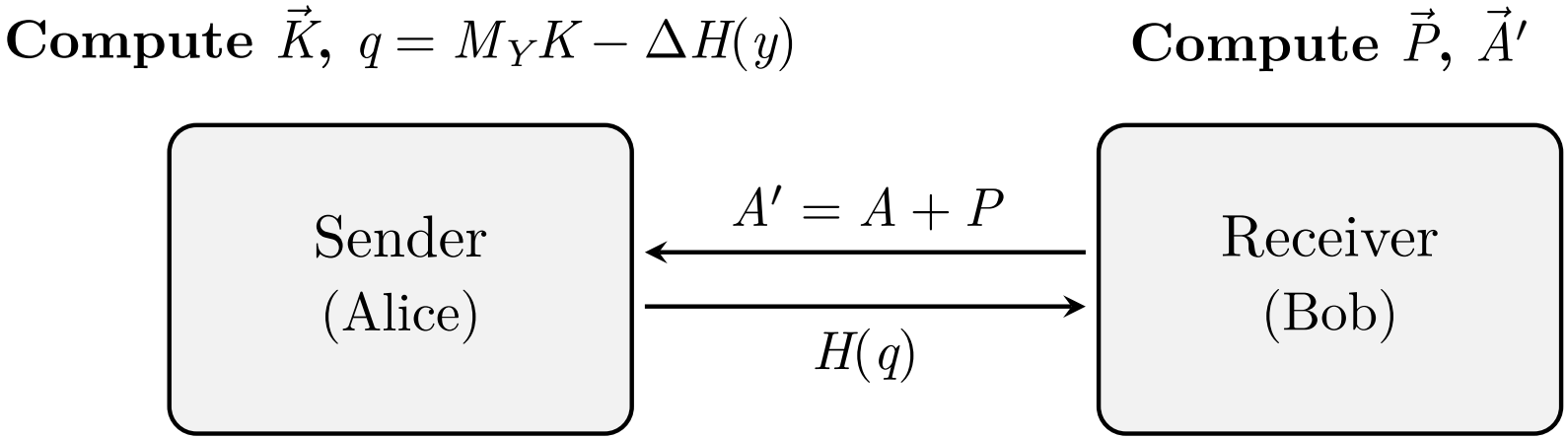
- Use $H(y)$ instead of 0 as the value,so $\mathbf{M_y}P{=}H(y)$. Alice has $F_{\mathbf{k}}(y) = H(\mathbf{M}_y\mathbf{k} - \Delta H(y))$.
- Bob can't konw the $H(y)$, so he can't get K.

OPRF



- Bob computes $M_x P = \{H(x_1), ..., H(x_n)\}$ and use $H(M_x C)$ to be OPRF result.
- Alice computes $K = \Delta A' + B$

OPRF



- Alice computes $M_Y K - \Delta H(y) = M_Y C + \Delta M_Y P - \Delta H(y)$
- if $y = x$,then $M_y = M_x$ and $H(y) = H(x)$,so the OPRF result is $H(M_y K - \Delta H(y)) = H(M_x C)$.

OKVS

Oblivious key value store:

- key-value: $\{(k_1, H(x_1)), ..., (k_n, H(x_n))\}$
- if $k' = k_1$ gets $H(x_1)$; else gets random value

Encode input sets $|X| = n$ and values to a vector $\vec{P} \in F^m$:

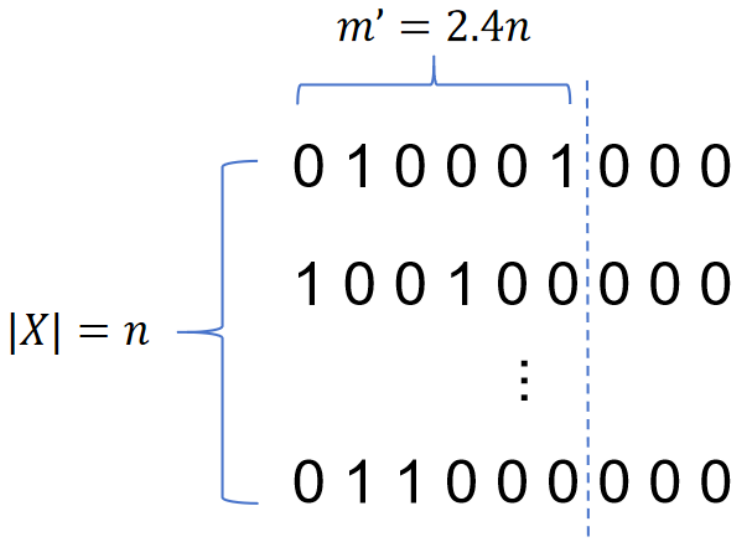
- sloves $M\vec{P}^\top = (H(x_i)), ..., H(x_n)$ to get $\vec{P}$.

Decode:

- Decode$(M_i, \vec{P})$=$H(x_i)$

PaXoS [PRTY20]

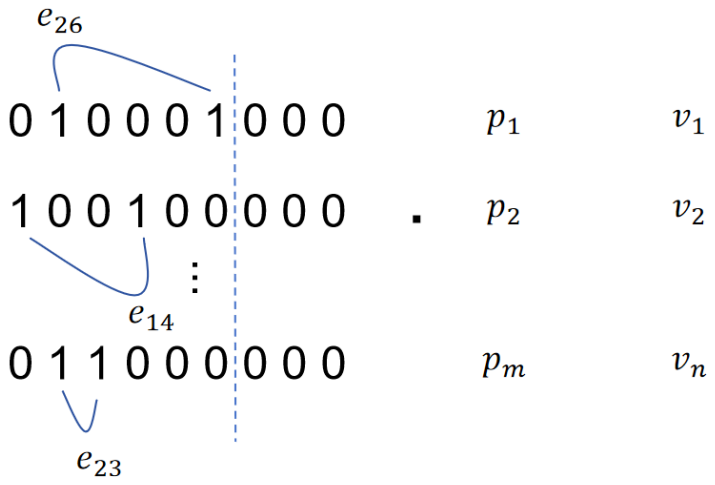
$M' \in \{0,1\}^{n \times m}$ be the submatrix formed by the first $m = 2.4n$ columns of M. As such, each row of M' has weight 2.



- Take those with a value of **1** as vertices and connect them with an edge $e_{j,k}$.

PaXoS

We have $M_i P = v_i$, where P is obtained by solving the system of equations.



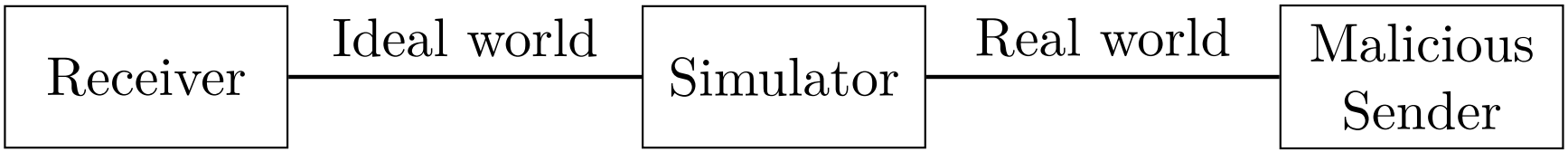
- Cycle: Use Gaussian elimination to solve the constraints.
- Tree: Assign values during the process of traversing tree nodes.

PaXoS

If the malicious Receiver wants to forge a $y \notin X$ after obtaining P:

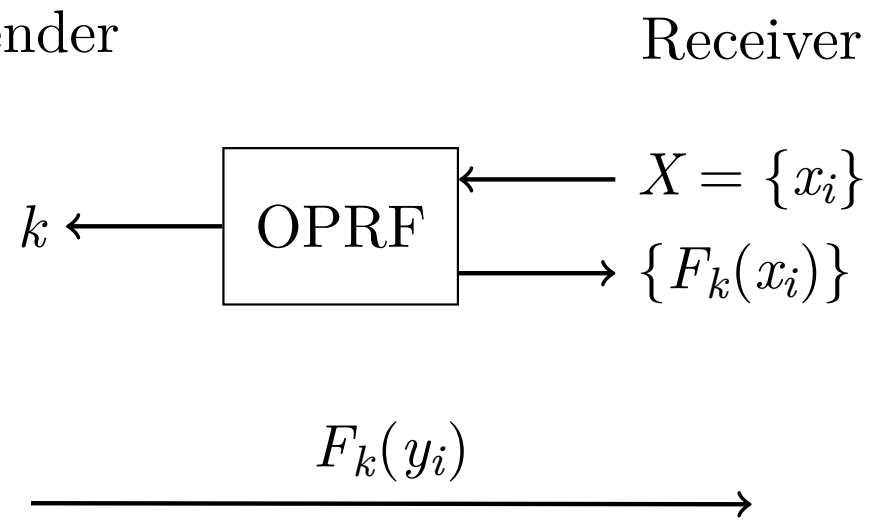
- For a certain $y \notin X$, and the i-th and j-th positions of the encoded string are 1, that is $P[i] \oplus P[j] = H(x)$.
- $H(\cdot)$ is a random oracle $\{0,1\}^* \to \{0,1\}^{O(\kappa)}$ bits.
- $\mathbf{Pr}(P[i] \oplus P[j] = H(x)) = \frac{1}{2^\kappa}$

Malicious Sender



- Ability: Simulator observes the interactions in the real protocol (including the Sender's queries to the OPRF and the final results sent by the Sender to the Receiver).
- Task: It must extract a set from it as the "Sender's input" and hand it over to the ideal function.
- Tool: Random oracle. Malicious sender cannot directly perform hash locally. It must ask RO and this process can be observed by Simulator.

Malicious Sender



- The malicious sender: forging OPRF values $F_k(y')$ that $y' \notin Y$.
- It is necessary to ensure that no collision occurs $y' \neq y;\ F_k(y') = F_k(y)$.This needs $\{0,1\}^{out}; out = 2\kappa$.

Malicious Sender

We want to obtain the intersection, and thus:

- $y_1, y_2 \notin X$:It's completely fine if a collision occurs.
- $y_1, y_2 \in X$:It still does not affect the obtained result.
- $y_1 \in X, y_2 \notin X$: The collision will cause problems.

If the simulator extracts y_2 as the Sender's input based on $F_k(y)$, the obtained intersection result will be incorrect.

The probability is:

$$\begin{aligned}\Pr\left[F(y_2) \in \{F(y) \mid y \in Y\} \ \wedge \ y_2 \notin X\right] &= 2^{-\text{out}} \cdot n_X \\ &= 2^{-\text{out}+\log_2(n_X)}\end{aligned}$$

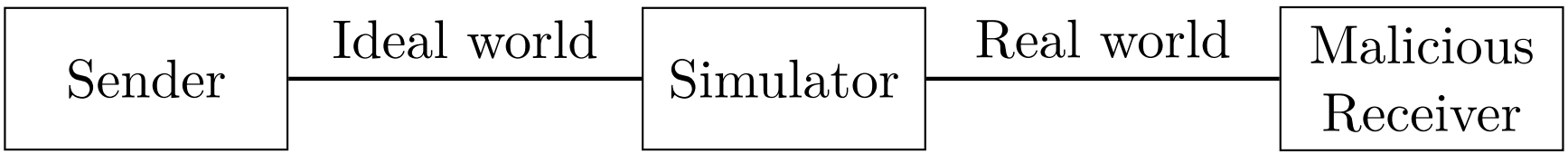
Malicious Sender

Using the union bound:

$$\begin{aligned}\Pr[\text{collision occurs}] &\leq 2^{-\text{out}+\log_2(n_X n_Y)} \\ &= O\big(2^{-\text{out}+\log_2(\kappa)+\log_2(n_Y)}\big).\end{aligned}$$

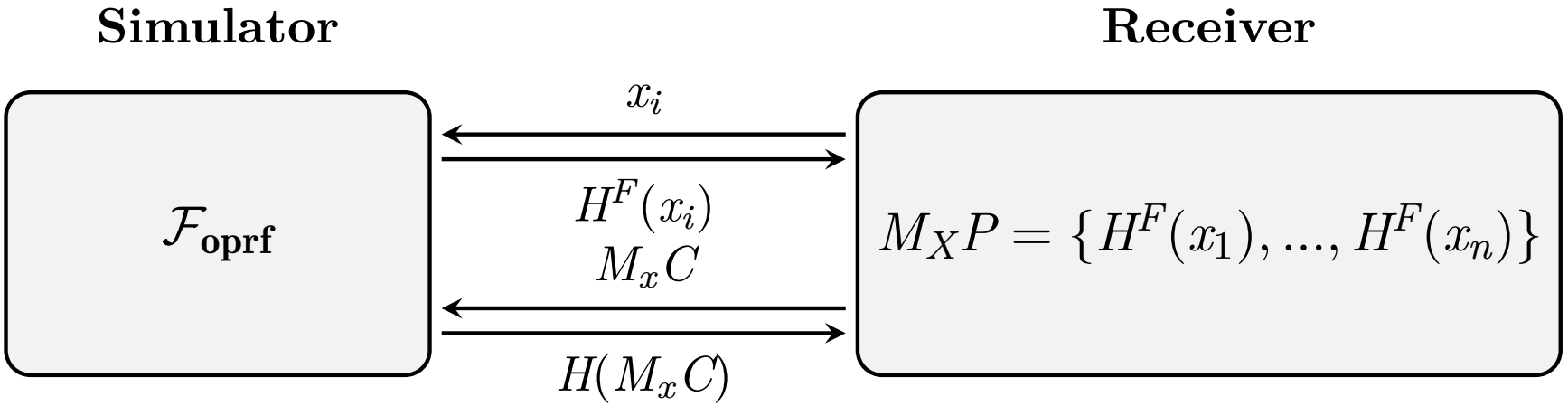
- Therefore, when the output length out $= \kappa$ (security parameter), the collision probability becomes negligible.
- If $y \in X \cap Y$, no collision would occur.
- The simulator can extract all elements y for which $F_k(y)$ has not collided as the Sender's set.

Malicious Receiver



- The simulator needs to act as the $\mathcal{F}_{\mathbf{oprf}}$ and return the intersection to the Receiver.
- It also needs to extract a set from it as the "Receiver's input" and hand it over to the ideal function.
- Still using RO.

Malicious Receiver



- Receiver needs P to build C and $H^F(x_i)$ to build P.
- Simulator can check $xP = H(x)$, if so adds x to set X'.
- Simulator sends the true set X' to Ideal OPRF and gets $H(x')$.
- Simulator programs $H(x'\,C) = H(x')$ for $x' \in X'$.

Malicious Receiver

- Simulator sends X' to $\mathcal{F}_{\mathbf{psi}}$ and gets $Z = X' \cap Y$.
- Simulator computes $\{F(z)|z \in Z\}$ along with $n_X - |Z|$ uniform values to acts as truely Receiver's set $F(x)$.